



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Ejercicio Carta ASM (Máquina de Estados)

Silverio Martínez Andrés

Lab. Organización y Arquitectura de
Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 3 de mayo de 2026

PRÁCTICA 13

DISEÑO DE UNA CARTA ASM

OBJETIVO:

El alumno comprenderá utilizando lenguaje VHDL, el funcionamiento y manejo de cartas ASM para la creación de máquinas de estados, las cuales controlan sistemas secuenciales con gran diversidad de salidas.

ESPECIFICACIONES:

Diseñar un contador que cuente en forma ascendente/descendente los números binarios del cero al tres, cuando el valor de la variable A sea uno la cuenta se efectuará de forma ascendente y si el valor de la variable A es cero la cuenta se efectuará en forma descendente. El cambio de cuenta de ascendente a descendente y viceversa se puede efectuar en cualquier estado. El diagrama de bloques se muestra en la figura 13.1 y en la figura 13.2 se muestra su carta ASM.

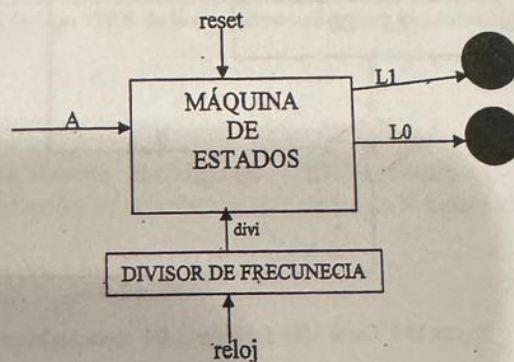
DIAGRAMA DE BLOQUES:

Figura 13.1. Diagrama de bloques del control del contador binario

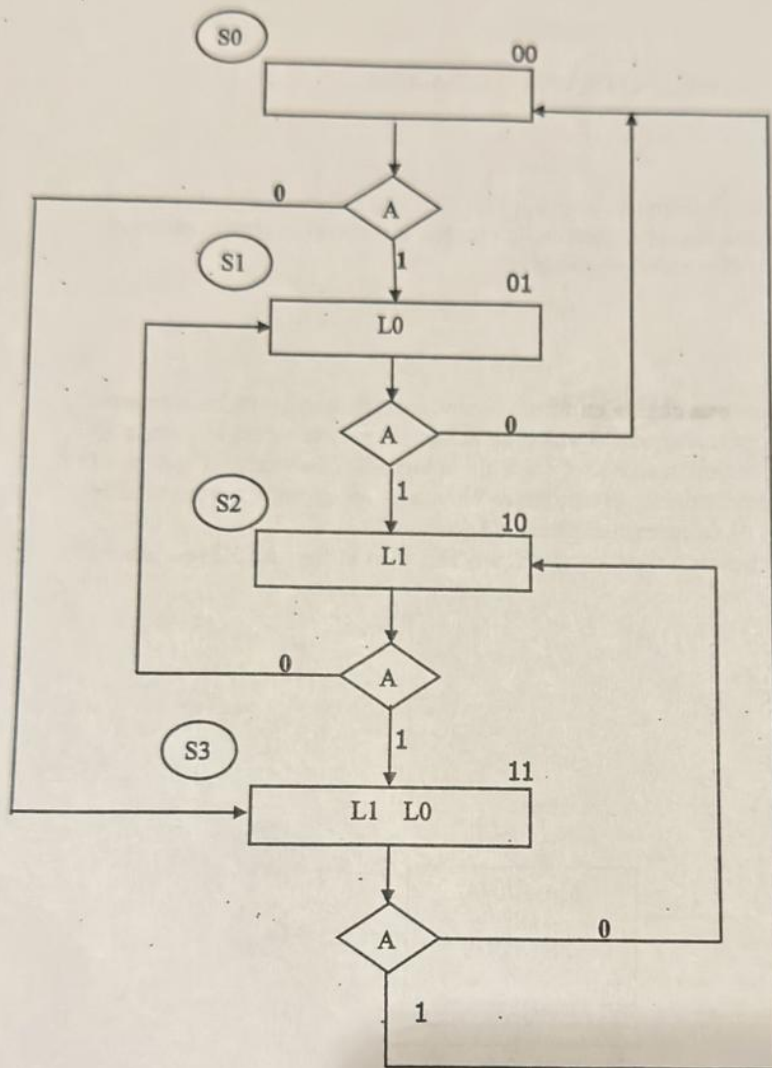


Figura 13.2 Carta ASM del control del contador binario.

En la figura 13.3 se describe el código en VHDL requerido para la entidad de este contador binario.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ASM is
  Port (RELOJ : in STD_LOGIC;
        RESET : in STD_LOGIC;
        A : in STD_LOGIC;
        L : out STD_LOGIC_VECTOR (1 downto 0));
end ASM;
```

Figura 13.3. Código VHDL de la entidad requerida para este contador binario

En la figura 13.4 se observa el código en VHDL requerido para la arquitectura de este contador binario y su simulación se muestra en la figura 13.5

```

architecture Behavioral of ASM is
    type ESTADOS is (s0,s1,s2,s3);
    signal epresente, esiguiente :ESTADOS;
begin
    process (reloj,reset)
    begin
        if reset='1' then epresente <= s0;
        elsif rising_edge (reloj) then
            epresente <= esiguiente;
        end if;
    end process;

    process (epresente, A)
    begin
        case epresente is
            when s0 =>
                L <= "00";
                if A='1' then
                    esiguiente<= s1;
                else
                    esiguiente <= s3;
                end if;
            when s1 =>
                L <= "01";
                if A='1' then
                    esiguiente<= s2;
                else
                    esiguiente<= s0;
                end if;
            when s2 =>
                L <= "10";
                if A='1' then
                    esiguiente<= s3;
                else
                    esiguiente <= s1;
                end if;
            when s3 =>
                L <= "11";
                if A='1' then
                    esiguiente<= s0;
                else
                    esiguiente<= s2;
                end if;
        end case;
    end process;
end Behavioral;

```

- Definición de los estados a usar

- Definición de las señales para interconectar procesos

- Proceso que define la transición de estados

Figura 13.4. Código VHDL de la arquitectura requerida para este contador binario

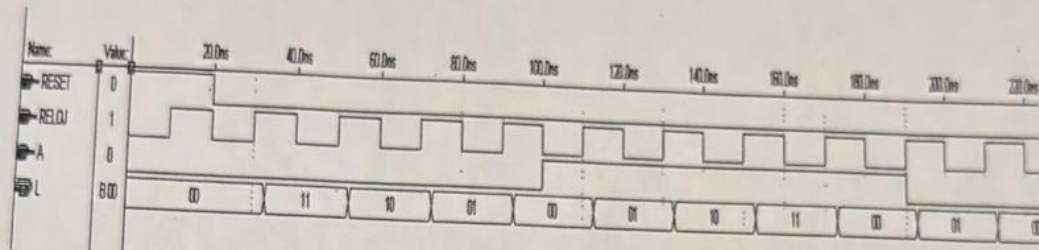


Figura 13.5. Simulación de este contador binario

REFERENCIAS

- [1] N..Chávez y J. Valeriano "Lenguaje de descripción de hardware VHDL"
- [2] N..Chávez "Dispositivos Lógicos Programables VLSI".

Ejercicio Carta ASM (Máquina de Estados)

CÓDIGO:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity ASM is
Port (
RELOJ : in STD_LOGIC;
RESET : in STD_LOGIC;
A : in STD_LOGIC;
L : out STD_LOGIC_VECTOR (1 downto 0)
);
end ASM;

architecture Behavioral of ASM is
-- Definición de los estados a usar
type ESTADOS is (s0, s1, s2, s3);
-- Definición de las señales para interconectar procesos
signal epresente, esiguiente : ESTADOS;

begin

-- Proceso que define la transición de estados
process (reloj, reset)
begin
if reset = '1' then
epresente <= s0;
elsif rising_edge (reloj) then
epresente <= esiguiente;
end if;
end process;

-- Proceso de lógica de estado siguiente y salida
process (epresente, A)
begin
case epresente is
when s0 =>
L <= "00";
if A = '1' then
esiguiente <= s1;
else
```

```

        esiguiente <= s3;
    end if;

    when s1 =>
        L <= "01";
        if A = '1' then
            esiguiente <= s2;
        else
            esiguiente <= s0;
        end if;

    when s2 =>
        L <= "10";
        if A = '1' then
            esiguiente <= s3;
        else
            esiguiente <= s1;
        end if;

    when s3 =>
        L <= "11";
        if A = '1' then
            esiguiente <= s0;
        else
            esiguiente <= s2;
        end if;
    end case;
end process;

end Behavioral;

```

Este código implementa un **contador binario reversible de 2 bits** utilizando una **Máquina de Estados Finitos (FSM)** de tipo **Moore**.

1. Arquitectura de la Máquina

El **diseño** se divide en dos procesos principales que trabajan en conjunto:

- **Proceso Secuencial (Memoria de Estado):** Gestiona los cambios de estado sincronizados con el flanco de subida del reloj (`rising_edge`). También incluye un **reset asíncrono** que devuelve el sistema al estado inicial `s0`.
- **Proceso Combinacional (Lógica de Transición y Salida):** Determina cuál será el siguiente estado y qué valor mostrará la salida según el estado actual y la entrada de control.

2. Lógica de Control (Señal "A")

La señal de entrada **A** actúa como el selector de dirección del conteo:

- **Si A = '1' (Conteo Ascendente):** La secuencia sigue el orden s0 s1 s2 s3 s0 (0, 1, 2, 3 en binario).
- **Si A = '0' (Conteo Descendente):** La secuencia se invierte s0 s3 s2 s1 s0 (0, 3, 2, 1 en binario).

3. Salidas (Naturaleza Moore)

La salida **L** es una función exclusiva del estado presente:

- **s0:** 00
- **s1:** 01
- **s2:** 10
- **s3:** 11

COMPILACIÓN:

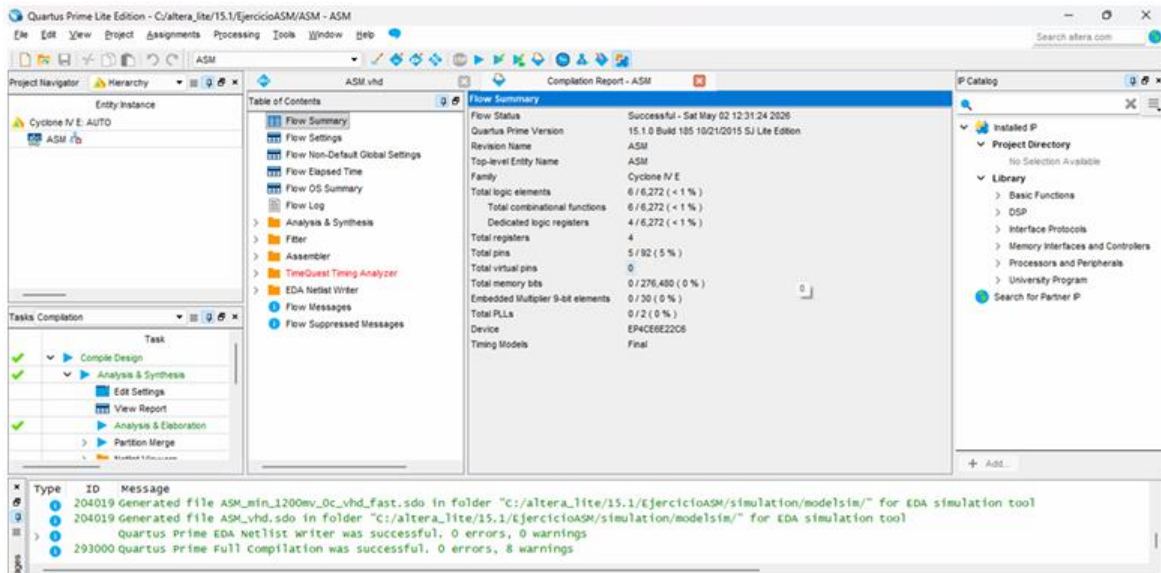
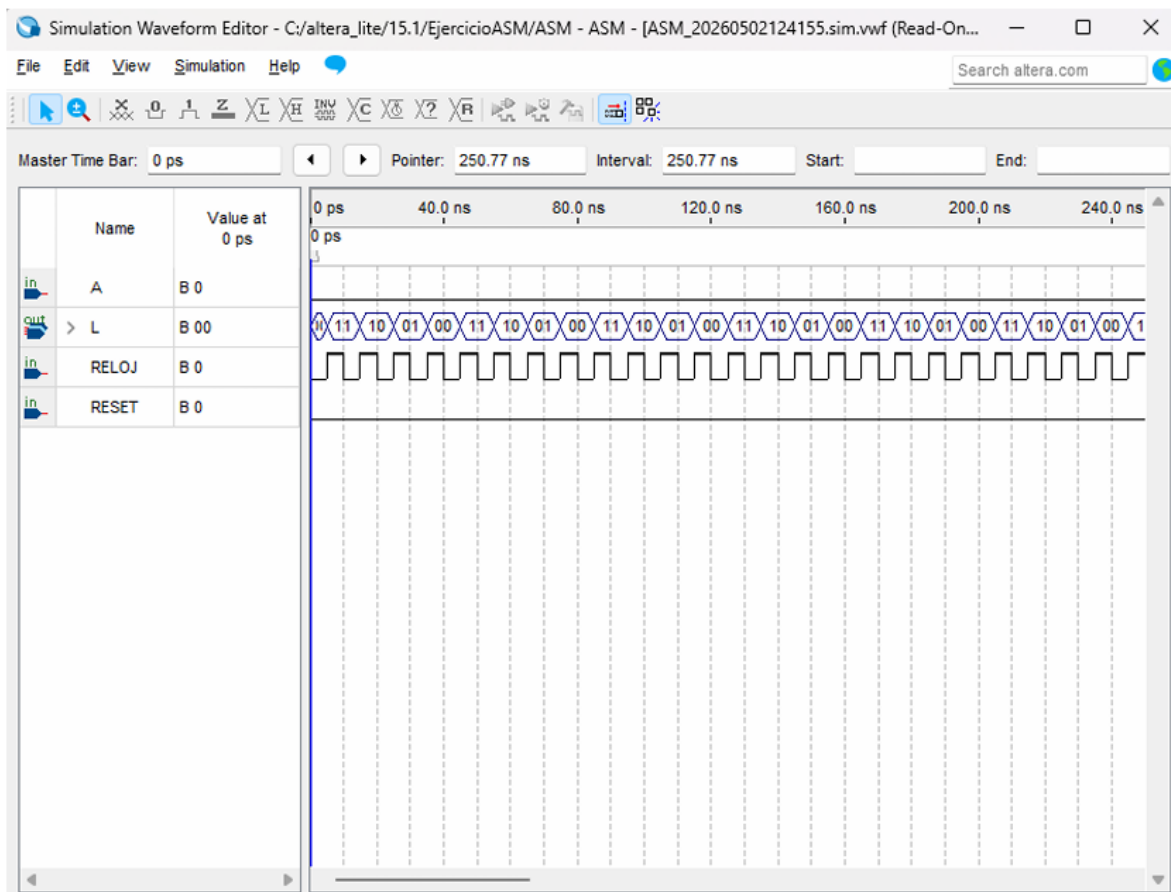


Imagen: Compilación

SIMULACIÓN



La simulación en el editor de formas de onda valida el comportamiento lógico del contador binario diseñado mediante la metodología de Cartas ASM. Al configurar una señal de reloj con un periodo de 10 ns y un ciclo de trabajo del 50%, se confirma que las transiciones de estado ocurren de forma sincrónica en cada **flanco de subida** (*rising edge*), corroborando la correcta implementación del proceso de asignación de estados en el código VHDL.

Como se observa en la captura, con la entrada de control '**A**' en nivel bajo ('0'), el sistema ejecuta la transición descendente definida en la carta ASM: partiendo de s0 hacia s3, y así sucesivamente. Este comportamiento ratifica la naturaleza de una **Máquina de Moore**, donde la salida **L** depende exclusivamente del estado actual. En consecuencia, la salida se estabiliza en el valor binario correspondiente al bloque de estado activo inmediatamente después de que el flanco de reloj actualiza el registro. La concordancia entre la secuencia observada (00, 11, 10, 01) y los tiempos de ejecución garantiza el cumplimiento de los requerimientos de lógica secuencial y temporización del contador reversible.